



Operators

➤ What Are Operators in Python?

- **Operators** are symbols that perform operations on variables and values.
- In Python, **operators are implemented as special methods** (e.g., $a + b \rightarrow a.__add__(b)$).

- **Classified into:**

- ❖ Arithmetic

- ❖ Logical

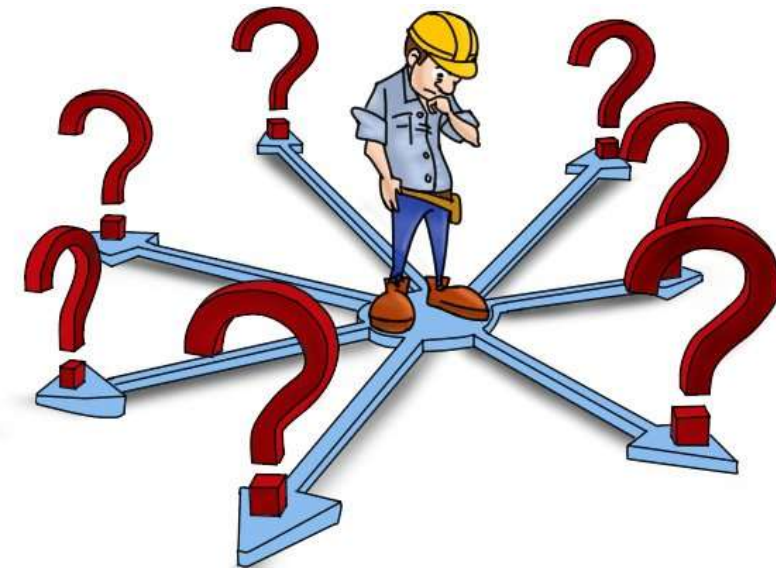
- ❖ Identity

- ❖ Membership

- ❖ Comparison

- ❖ Assignment

- ❖ Bitwise



Arithmetic Operators

Arithmetic operators in Python are used to perform **basic mathematical operations** between numeric values. They work with int, float, complex, and even with objects that define these operations using special methods.

Operator	Symbol	Internal Method	Description
Addition	+	<code>__add__()</code>	Adds the right operand to the left one. Useful for summing values.
Subtraction	-	<code>__sub__()</code>	Subtracts the right operand from the left one. Common in difference calculations.
Multiplication	*	<code>__mul__()</code>	Multiplies both operands. Frequently used in scaling, quantity calculations, etc.
Division	/	<code>__truediv__()</code>	Divides the left operand by the right and returns a floating-point result.
Floor Division	//	<code>__floordiv__()</code>	Performs division and returns the integer part of the result (rounded down).
Modulus	%	<code>__mod__()</code>	Returns the remainder after division. Useful in cyclic or even/odd conditions.
Exponentiation	**	<code>__pow__()</code>	Raises the left operand to the power of the right. Used in scientific calculations.

➤ Arithmetic Operators

✓ Example Code:

```
hello.py  X
hello.py
1  a = 15
2  b = 4
3  # Output
4  print(a + b)  # 19
5  print(a - b)  # 11
6  print(a * b)  # 60
7  print(a / b)  # 3.75
8  print(a // b) # 3
9  print(a % b)  # 3
10 print(a ** b) # 50625
```

📖 Important Notes:

- / always returns a **float**, even if both numbers are integers.
- // is ideal for **loop counters**, **index calculations**, or scenarios where **whole numbers** are required.
- % is widely used in **modulo arithmetic**, **validations**, and **alternating logic**.
- ** is more readable than using the `math.pow()` function.



➤ When to Use Arithmetic Operators

📖 Arithmetic operators are not limited to mathematical calculations they are crucial in a wide range of **real-world software systems**, making logic simpler and more expressive.

1. 📊 Data Processing

- Performing statistical calculations like **mean, median, standard deviation**.
- Useful in spreadsheet-like operations using Pandas or NumPy.
- Example: $\text{average} = \text{total} / \text{count}$

2. ☐ Billing and Pricing Systems

- Automatically compute totals, discounts, taxes, and balances.
- Arithmetic is the foundation for all **financial software**.
- Example: $\text{final_price} = (\text{price} * \text{quantity}) - \text{discount}$

3. 📈 Analytics and Scientific Computation

- Used in formulas for **data science**, engineering, simulations, etc.
- Example: $\text{velocity} = \text{distance} / \text{time}$

4. 🎮 Game Development

- Handles **score updates**, **player movement**, **timing logic**.
- Physics-based games rely heavily on $+$, $-$, $*$, $/$, and $**$.
- Example: $\text{new_x} = \text{x} + (\text{speed} * \text{time})$



➤ Logical Operators

📖 Logical operators are used to **combine conditional statements** (also called Boolean expressions).

These operators return either True or False based on the logic applied.

In Python, the main logical operators are:

- and
- or
- not

They are essential in **decision-making, control flow, and boolean logic operations.**

🔧 Logical Operators Table

Operator	Syntax	Description	Example	Result
and	x and y	Returns True if both operands are True	True and False	False
or	x or y	Returns True if at least one operand is True	True or False	True
not	not x	Returns True if the operand is False, and vice versa	not True	False



➤ Logical Operators

□ Truth Table:

✓ and Operator

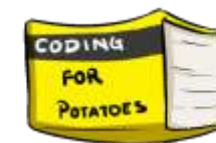
A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

✓ or Operator

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

✓ not Operator

A	not A
True	False
False	True



➤ ! Use Cases of Logical Operators

1. 🔁 Control Flow (if-else, while loops):

- Combine multiple conditions:

```
if age >= 18 and has_id:  
    print("Eligible to vote")
```

2. 🔍 Filtering Data:

- Used in list comprehensions or filtering functions.

```
[x for x in data if x > 10 and x < 50]
```

3. 🗝️ Authentication Systems:

- Check multiple credentials:

```
if username == "admin" and password == "1234":  
    print("Access Granted")
```

4. ⬛ Safety Checks and Validations:

- Avoiding divide by zero or null errors:

```
if player_health > 0 or lives > 0:  
    continue_game()
```

5. 🎮 Game Logic:

- Conditions for win/lose or game state:

```
if denominator != 0:  
    result = numerator / denominator
```



➤ ✓ Example Code:

```
hello.py x
hello.py
1  x = 5
2  y = 10
3  # AND
4  print(x > 0 and y > 0)    # True
5  # OR
6  print(x < 0 or y > 0)    # True
7  # NOT
8  print(not (x > y))        # True (since x > y is False)
```

□ Tip for Learners:

- Use parentheses () for **clarity and grouping** complex conditions.
- Python evaluates logical expressions using **short-circuit evaluation**:
 - In x and y, if x is False, Python **won't evaluate y**
 - In x or y, if x is True, Python **won't evaluate y**



➤ Identity Operators

📖 Identity operators are used to **compare memory locations** of two objects, i.e., whether both operands **refer to the same object** (same memory address), not just equal values.

Operator	Syntax	Description
is	x is y	Returns True if x and y refer to the same object
is not	x is not y	Returns True if x and y do not refer to the same object

☐ Use Cases:

- Comparing references in **memory**.
- Checking if two variables are pointing to the **same instance** (useful in class objects).
- Often used with None:

```
if result is None:
    print("No result found")
```



➤ ✓ Example: Identity Operators

```
hello.py
1  a = [1, 2, 3]
2  b = a
3  c = [1, 2, 3]
4  print(a is b)      # True (both point to same object)
5  print(a is c)      # False (same content but different objects)
6  print(a == c)      # True (values are equal)
7  print(a is not c)  # True
8
```



➤ Membership Operators

📖 Membership operators are used to **check if a value exists in a sequence** like list, string, tuple, or dictionary.

Operator	Syntax	Description
in	x in seq	Returns True if x exists in the sequence seq
not in	x not in seq	Returns True if x does not exist in seq

✓ Example: Membership Operators

```

🐍 hello.py
1  fruits = ["apple", "banana", "mango"]
2  text = "Python is awesome"
3
4  print("apple" in fruits)           # True
5  print("grape" not in fruits)      # True
6  print("awesome" in text)          # True
7  print("java" in text)             # False

```



➤ ⚖️ Comparison Table: Identity vs Membership

Feature	Identity (is, is not)	Membership (in, not in)
Checks	Object identity (memory address)	Presence in a sequence or collection
Compares	References	Elements or substrings
Used with	Variables, objects	Lists, strings, tuples, dicts, sets
Returns	True or False	True or False

□ Use Cases:

- Checking presence of keys, values, characters in:
 - Lists, strings, sets, tuples, dictionaries
- Used in **searching, filtering, and conditional logic**.

• **Example:**

```
if "admin" in user_roles:
    print("Show Admin Panel")
```

✓ Bonus Tip:

- Use `==` to compare **values**, `is` to compare **identity (reference)**.
- For **custom objects**, you can override `__eq__()` for value comparison and `__hash__()` for usage in sets/dicts.



Bitwise Operators

 Bitwise operators are used to **perform operations on individual bits** of integers. These are fast and commonly used in **low-level programming, device control, data compression, and encryption algorithms.**

Python automatically converts integers to binary for bitwise operations.

☐ **Binary Examples:**

Decimal	Binary
5	0101
3	0011





Bitwise Operator Table

Operator	Symbol	Description	Example	Result (binary)
AND	&	Sets each bit to 1 if both bits are 1	$5 \& 3 \rightarrow 101 \& 011$	$001 \rightarrow 1$
OR		Sets each bit to 1 if one or both bits are 1	$5 3$	$111 \rightarrow 7$
XOR	^	Sets bit to 1 if only one bit is 1	$5 \wedge 3$	$110 \rightarrow 6$
NOT	~	Inverts all bits (1's complement)	~ 5	$-(5+1) \rightarrow -6$
Left Shift	<<	Shifts bits to the left (adds zeros to right)	$5 << 1$	$1010 \rightarrow 10$
Right Shift	>>	Shifts bits to the right (discards bits on right)	$5 >> 1$	$10 \rightarrow 2$



Truth Tables:

✓ **AND (&)**

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

✓ **OR (|)**

A	B	A B
0	0	0
0	1	1
1	0	1
1	1	1

✓ **XOR (^)**

A	B	A ^ B
0	0	0
0	1	1
1	0	1
1	1	0



➤ Practical Use Cases:

- ❑ **Hardware-Level Programming:**
 - Set, clear, toggle bits in device control registers.
- 🔒 **Cryptography:**
 - XOR used in encryption algorithms for scrambling data.
- 📦 **Data Compression:**
 - Efficient bit manipulation for compact storage.
- 🎮 **Game Development:**
 - Manage flags (like collision states, character abilities).
- 🏗️ **Performance Optimization:**
 - Use shifts for fast multiplication/division by powers of 2:

```
x << 3 # x * 8
x >> 2 # x // 4
```

❑ **Tips for Learners:**

- Always convert to binary for clarity when practicing.
- Bitwise NOT (~) returns -(n+1) in Python due to 2's complement representation.
- Use `bin()` to see binary representations:

```
print(bin(5)) # 0b101
```



➤ Comparison Operators

📖 Comparison operators in Python are used to **compare values** and return a Boolean result (True or False). These are fundamental in **conditional statements**, **loops**, and **decision-making** logic.

They can be used with **numbers**, **strings**, **lists**, **tuples**, and even **custom objects** (if comparison methods are defined).

☐ Comparison Operator Table

Operator	Symbol	Description	Example	Result
Equal to	==	Returns True if both values are equal	5 == 5	True
Not equal to	!=	Returns True if values are not equal	5 != 3	True
Greater than	>	Returns True if left > right	7 > 4	True
Less than	<	Returns True if left < right	2 < 9	True
Greater or equal to	>=	Returns True if left ≥ right	6 >= 6	True
Less or equal to	<=	Returns True if left ≤ right	3 <= 5	True



➤ Example of Comparison Operators

✓ Example Code:

□ Use with Other Data Types:

```
hello.py
1  x = 10
2  y = 20
3
4  print(x == y)    # False
5  print(x != y)    # True
6  print(x > y)     # False
7  print(x < y)     # True
8  print(x >= 10)   # True
9  print(y <= 25)   # True
10 |
```

```
hello.py
1  # Strings
2  print("apple" == "Apple")  # False (case-sensitive)
3  print("cat" < "dog")       # True (alphabetical order)
4
5  # Lists
6  print([1, 2] == [1, 2])    # True
7  print([1, 2] != [2, 1])   # True
```

□ Python compares elements **lexicographically** (like dictionary order) for sequences.



➤ ! Use Cases of Comparison Operators

1. 🔁 Conditional Logic (if-else):

```
if age >= 18:  
    print("Eligible to vote")
```

3. 🎮 Game Scoring Systems:

```
if score == high_score:  
    print("New Record!")
```

1. 📊 Data Filtering:

```
numbers = [1, 4, 8, 10]  
filtered = [x for x in numbers if x > 5]
```

4. ✔ Input Validation:

```
if len(password) < 8:  
    print("Password too short")
```

2. 🔁 Loops with Conditions:

```
while count <= 5:  
    print(count)  
    count += 1
```

❑ Tips

- Use == for **equality**, not = (assignment).
- Avoid comparing different data types unless necessary:
`5 == "5" # False`
- Combine with logical operators (and, or) for complex conditions.



➤ Assignment Operators

📖 Assignment operators in Python are used to **assign values to variables**. They can also **combine assignment with arithmetic or bitwise operations**, simplifying expressions and making the code cleaner.

Python supports **simple** as well as **compound assignment operators**.

✓ Example Code:

```
hello.py
1  x = 10
2
3  x += 5    # x = 15
4  x -= 3    # x = 12
5  x *= 2    # x = 24
6  x /= 4    # x = 6.0
7  x **= 2   # x = 36.0
```



Assignment Operator Table

Operator	Symbol	Example	Equivalent To	Description
Assign	=	x = 5	—	Assigns the value 5 to variable x
Add & assign	+=	x += 3	x = x + 3	Adds 3 to x and reassigns the result
Subtract & assign	-=	x -= 2	x = x - 2	Subtracts 2 from x
Multiply & assign	*=	x *= 4	x = x * 4	Multiplies x by 4
Divide & assign	/=	x /= 2	x = x / 2	Divides x by 2 (float result)
Floor divide & assign	//=	x //= 3	x = x // 3	Integer division and reassignment
Modulus & assign	%=	x %= 5	x = x % 5	Stores the remainder of x ÷ 5
Exponent & assign	**=	x **= 2	x = x ** 2	Raises x to the power of 2
Bitwise AND & assign	&=	x &= 1	x = x & 1	Performs bitwise AND
Bitwise OR & assign	=	x = 2	x = x 2	Performs bitwise OR
Bitwise XOR & assign	^=	x ^= 3	x = x ^ 3	Performs bitwise XOR
Left shift & assign	<<=	x <<= 1	x = x << 1	Left shifts x by 1 bit
Right shift & assign	>>=	x >>= 1	x = x >> 1	Right shifts x by 1 bit



➤ Use Cases of Assignment Operators:

1. Data Manipulation in Loops

```
total = 0
for i in range(1, 6):
    total += i  # Adds 1 + 2 + ... + 5
```

2. Decreasing or Scaling Values

```
discount_price = price
discount_price *= 0.9  # Apply 10% discount
```

3. Bitwise Flag Updates

```
flags = 0b1010
flags |= 0b0001  # Set a specific bit
```

4. Scorekeeping in Games

```
score = 0
score += 10  # Add points for an action
```

☐ Tips:

- Use compound assignments to **reduce redundancy** and **improve readability**.
- Be cautious with `/=` — it always returns a float, even if both operands are integers.
- Combine with loops or conditionals for dynamic updates.



